

Sports Tournament Knowledge Graph

Project Report

Submitted in partial fulfillment of requirements to

Computer Science Honors (CSHK – SP25)

IV/IV B.Tech (VII Semester)

By

Tulam Sai Sudheer (Y22CS184)



October, 2025

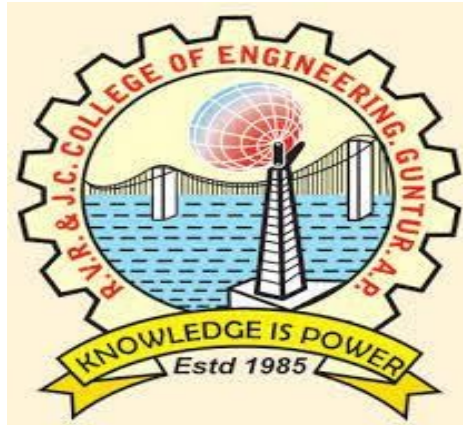
R.V.R. & J.C. COLLEGE OF ENGINEERING (Autonomous)

(Approved by AICTE, Affiliated to Acharya Nagarjuna University)

Chandramoulipuram::Chowdavaram,

GUNTUR-522019

R.V.R&J.C. COLLEGE OF ENGINEERING(Autonomous)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that this Knowledge Graphs Project Report titled “**Sports Tournament Knowledge Graph**” is the project conducted by **Tulam Sai Sudheer (Y22CS184)** and submitted in partial fulfillment of the requirements to CSHK – SP25 Knowledge Graphs during the Academic Year 2025-2026.

Dr. M. Sreelatha

Lecturer Incharge

Dr. M. Sreelatha

Prof.&Head,CSE

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without a proper suggestions, guidance and support. Combination of these factors acts like backbone to our Knowledge Graphs Project titled “**Sports Tournament Knowledge Graph.**” I would like to express my sincere gratitude to **Dr. M. Sreelatha**, In-charge for our project. Her expertise, guidance, and support were instrumental in the successful completion of this Project.

I express my sincere thanks to **Dr. M. Sreelatha**, Head of the Department of Computer Science and Engineering for her encouragement, support, commitment to enhance project experience.

I am very much thankful to **Dr. Kolla Srinivas**, Principal of R.V.R &J.C College of Engineering, Guntur for providing this supportive Environment and to engage in research activities.

Finally, I submit my sincere thanks to lab staff in Department of Computer Science and Engineering for their cooperation, support, for providing administrative support and technical assistance during selection.

TULAM SAI SUDHEER (Y22CS184)

ABSTRACT

We express our deepest gratitude to our project guide, faculty members, and institution for their constant encouragement, valuable insights, and unwavering support throughout the development of our project, **“Sports Tournament Knowledge Graph.”** Their guidance helped us understand the integration of semantic web concepts with modern AI technologies. We are especially thankful for the opportunity to explore how **RDF (Resource Description Framework)** and **SPARQL** can be applied to model and query real-world sports data in a structured and meaningful way. Their mentorship inspired us to combine theoretical knowledge with practical implementation effectively.

This project provided us with an in-depth understanding of knowledge representation and graph-based data modeling. By developing a system capable of answering natural language questions using **Google Gemini AI**, **Streamlit**, and **PyVis**, we learned how artificial intelligence can enhance user interaction with linked data. The project also helped us gain hands-on experience in building and visualizing RDF graphs representing entities like teams, players, matches, venues, and performance statistics. Overall, this project has significantly enriched our technical and analytical skills and strengthened our ability to apply AI and data science concepts to solve real-world problems.

CONTENTS

Title Page	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
Contents	v
List of Figures	vii
1. Introduction	1-2
1.1 Introduction to the Project	
1.2 Problem Definition	
1.3 Significance of the Work	
1.4 Objectives and Scope	
2. System Analysis	3-4
2.1 Requirements Specification	
2.2 Educational Domain Ontology	
3. System Design	5-9
3.1 Architecture of the Proposed System	
3.2 Database Schema Design	
3.3 Workflow of the Proposed System	
4. Implementation	10-11
4.1 Development Environment and Methodology	
4.2 Core Module Implementation	
4.3 Integration and Testing	
5. Result Analysis	12-16
5.1 System Demonstration	
○ Side Bar Navigation	

- Interface
- Demo-I
- Demo-II
- Demo-III
- RDF Graph Visualization

5.2 Performance Analysis

5.3 Feature Comparison & Feedback

6. Conclusion and Future Work

17-19

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.3.1	Work Flow	9
5.1.1	Side Navigation Bar	12
5.1.2	Interface	12
5.1.3	Demo-I	13
5.1.4	Demo-II	13
5.1.5	Demo-III	14
5.1.6	RDF Graph Visualization	15

1. Introduction

1.1 Introduction to the Project

The project introduces an intelligent Streamlit-based chatbot system that enables users to query and explore an RDF knowledge graph of IPL cricket data through natural language interaction. By integrating the Google Gemini generative AI model, the chatbot automatically converts user questions into SPARQL queries, executes them on the underlying RDF graph, and returns structured results related to players, teams, matches, and performances. In addition to answering queries, the application provides an interactive visualization of the RDF graph using PyVis, allowing users to understand relationships between entities in a graphical format. This system demonstrates how semantic web technologies and AI-driven natural language understanding can work together to simplify complex data access and enhance user engagement with structured datasets.

1.2 Problem Definition

The problem addressed in this project is the difficulty of querying and understanding complex cricket match data stored in RDF format using traditional SPARQL queries, which require technical expertise. Cricket analysts, students, and fans often struggle to extract meaningful insights from RDF datasets due to the steep learning curve of semantic web technologies. This project aims to solve that problem by developing an intelligent chatbot interface that allows users to ask natural language questions about IPL cricket matches and automatically converts them into SPARQL queries using Google's Gemini API. The system executes these queries on an RDF knowledge graph and presents the results in an easy-to-understand format, along with an interactive visualization of the RDF data for better comprehension.

1.3 Significance of the Work

The significance of this work lies in its ability to bridge the gap between complex semantic web technologies and user-friendly data access through natural language interaction. By integrating RDF-based knowledge representation with Google's Gemini-powered SPARQL query generation, the system enables non-technical users to retrieve meaningful cricket insights without requiring knowledge of query languages. This approach not only enhances data accessibility and usability but also demonstrates the potential of combining AI-driven natural language processing with semantic data models. The project serves as a valuable tool for sports analysts, researchers, and enthusiasts to explore, analyze, and visualize cricket data intuitively and interactively.

1.4 Objectives and Scope

The main objective of this project is to develop an intelligent, user-friendly system that allows users to query and visualize IPL cricket data stored in RDF format using natural language. The system aims to automatically convert user questions into SPARQL queries through the integration of Google's Gemini API, execute them on an RDF knowledge graph, and display the results in an interactive Streamlit interface. The scope of this work includes the design and implementation of a chatbot interface for natural language interaction, SPARQL query generation and execution, and RDF graph visualization for better understanding of relationships between cricket entities such as players, teams, matches, and venues. This project demonstrates the practical application of AI and semantic web technologies in simplifying complex data retrieval and enhancing sports data analytics.

2. System Analysis

2.1 Requirements Specification

2.1.1 Functional Requirements

The functional requirements of this project include the ability to accept natural language questions from users through a Streamlit-based chatbot interface, convert those questions into valid SPARQL queries using Google's Gemini API, and execute the generated queries on an RDF dataset containing IPL cricket information. The system must fetch and display query results in a structured, readable format such as tables using pandas DataFrames. Additionally, it should provide an interactive RDF graph visualization using PyVis to help users explore relationships between entities like players, teams, matches, and venues. The application should also maintain chat history, allow easy navigation between chatbot and visualization views, and offer sample questions for user guidance all while ensuring smooth query execution, error handling, and real-time interaction.

2.1.2 Non-Functional Requirements

The non-functional requirements of this project focus on ensuring performance, usability, reliability, and scalability of the system. The application should provide quick and accurate responses to user queries with minimal latency during SPARQL query generation and execution. It should have an intuitive and user-friendly interface that allows seamless interaction even for users without technical expertise. The system must be reliable, handling invalid inputs or query errors gracefully without crashing. It should maintain consistent performance while processing large RDF datasets and support scalability for future expansion of cricket data. Additionally, the application should ensure data integrity, maintain responsiveness across devices, and provide visually appealing graph visualizations for an enhanced user experience.

2.2 Educational Domain Ontology

1. Core Ontology

The core ontology defines the fundamental entities and relationships that form the foundation of the educational domain. It includes basic concepts such as Student, Teacher, Course, Subject, Department, and Institution. These entities are linked by essential relationships like enrollsIn, teaches, belongsToDepartment, and offersCourse. The core ontology ensures a consistent semantic structure and interoperability across different educational systems by defining universally understood terms and relationships.

2. Relationship Ontology

The relationship ontology specifies the types of associations and interactions between entities within the educational environment. It captures dynamic relationships such as Student–Course enrollment, Teacher–Course assignment, Department–Program affiliation, and Student–Assessment performance. This ontology supports reasoning over connections, enabling systems to infer relationships like “a student who enrolls in a course taught by a teacher belongs to the same department,” thus enhancing data linkage and contextual understanding.

3. Domain-Specific Business Rules Ontology

The domain-specific business rules ontology defines the policies, constraints, and operational rules governing educational processes. Examples include a student must complete prerequisites before enrolling in an advanced course, a teacher can teach multiple courses within a semester, and each department must offer at least one core subject. These rules ensure compliance with institutional guidelines and automate decision-making within educational systems. The ontology enables intelligent reasoning and validation of academic workflows, ensuring consistency and accuracy in managing educational data.

3. System Design

3.1 Architecture of the Proposed System

The architecture of the proposed system is designed in a modular and layered structure that integrates Natural Language Processing (NLP), Semantic Web, and Interactive Visualization components to deliver intelligent querying of cricket match data. The system consists of the following major layers:

1. User Interface Layer:

This layer is implemented using Streamlit, providing an intuitive and interactive chatbot interface for users to enter natural language queries. It includes two main views — the Chatbot Interface for question answering and the RDF Graph Visualization Interface for exploring the knowledge graph visually.

2. Query Processing Layer:

This layer handles the transformation of user queries into machine-understandable SPARQL queries. The user's input is sent to Google's Gemini API, which applies NLP techniques to interpret the question and generate a syntactically and semantically correct SPARQL query based on the RDF schema.

3. Knowledge Graph Layer:

The RDF Knowledge Graph, stored in a .ttl (Turtle) file, represents structured cricket match data, including matches, teams, players, venues, and performances. It is managed using RDFLib, which enables parsing, storing, and querying RDF data using SPARQL.

4. Execution Layer:

This layer executes the generated SPARQL queries on the RDF graph using RDFLib's query engine. It retrieves relevant results such as player statistics, match outcomes, or team details and formats them into a structured pandas DataFrame for display.

5. Visualization Layer:

The visualization layer uses PyVis to generate an interactive graphical view of the RDF dataset. This allows users to explore the entities and relationships within the cricket knowledge graph dynamically.

6. Integration and Communication Layer:

This layer ensures seamless communication between components — the chatbot, Gemini API, RDF graph, and visualization modules. It also manages data flow, error handling, caching (via `st.cache_resource`), and chat history maintenance within Streamlit.

3.2 Database Schema Design

The database schema of the proposed system is based on RDF (Resource Description Framework), which represents cricket match data in the form of triples — subject, predicate, and object. Instead of traditional relational tables, data is stored as interconnected nodes and relationships to form a Knowledge Graph. The schema captures entities such as Matches, Teams, Players, and Venues, along with their properties and interconnections.

1. Node Schemas

a) Match Node

Represents an IPL match and stores essential match-level information.

Properties:

- matchID
- date
- city
- venue
- teamA
- teamB

- winner
- totalRuns_TeamA
- totalRuns_TeamB
- RDF Type: vocab:Match

b) Team Node

Represents each participating team.

Properties:

- teamID
- teamName
- shortName
- RDF Type: vocab:Team

c) Player Node

Represents individual players and their attributes.

Properties:

- playerID
- playerName
- playsForTeam
- playerRole
- isStartingXI
- isImpactPlayerIn
- RDF Type: vocab:Player

d) Venue Node

Represents match venue details.

Properties:

- venueID
- name
- location
- RDF Type: vocab:Venue

e) BattingPerformance Node

Represents the batting statistics of a player in a specific match.

Properties:

- runs
- ballsFaced
- notOut
- didNotBat
- RDF Type: vocab:BattingPerformance

f) BowlingPerformance Node

Represents the bowling performance of a player in a specific match.

Properties:

- overs
- runsConceded
- wicketsTaken
- RDF Type: vocab:BowlingPerformance

Relationship Schemas

Relationship	Domain (Source Node)	Range (Target Node)	Description
vocab:teamA	Match	Team	Links Match to the first team
vocab:teamB	Match	Team	Links Match to the second team
vocab:winner	Match	Team	Connects Match to the winning team
vocab:venue	Match	Venue	Specifies where the match was played
vocab:playsForTeam	Player	Team	Connects player to their respective team
vocab:battingScore	Player	BattingPerformance	Links player to batting statistics
vocab:bowlingScore	Player	BowlingPerformance	Links player to bowling statistics
vocab:atMatch	BattingPerformance / BowlingPerformance	Match	Specifies the match for that performance

3.3 Workflow of the Proposed System

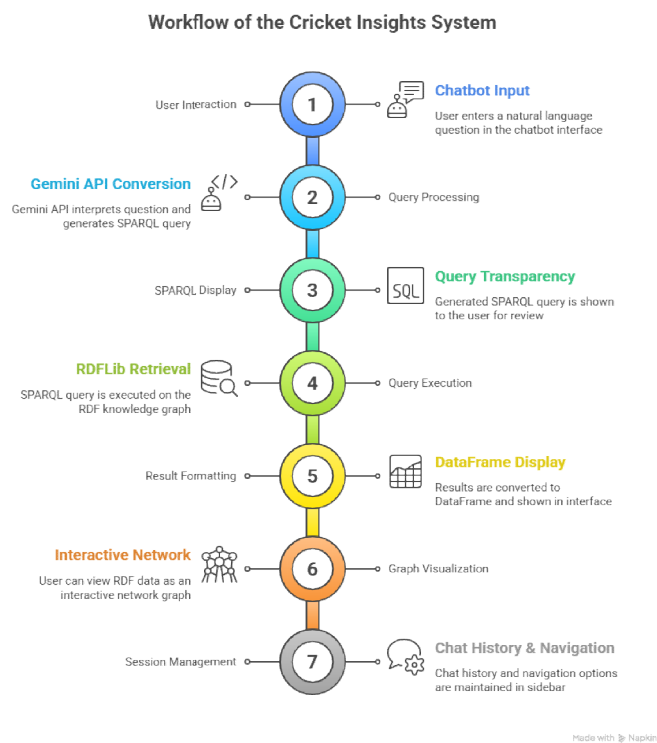


Fig: 3.3.1

4. Implementation

4.1 Development Environment and Methodology

4.1.1 Development Framework

The development of the Cricket Knowledge Graph Chatbot follows a modular, AI-integrated, and user-centric approach combining semantic web technologies with natural language processing and data visualization. The system is implemented using Python and Streamlit, leveraging Google's Gemini API for intelligent SPARQL query generation.

This provides a simple and efficient way to build interactive web applications in Python. The backend integrates the RDFLib library for parsing and querying RDF data, PyVis for network visualization, and pandas for structured data representation. The Google Generative AI (Gemini) API is used to interpret natural language questions and convert them into SPARQL queries automatically. The application runs in a local development environment using Python 3.10+, ensuring compatibility and scalability.

Tools and Technologies Used:

Frontend/UI: Streamlit

Backend: Python

AI Integration: Google Gemini API

Data Handling: RDFLib, pandas

Visualization: PyVis

Data Format: Turtle (.ttl) for RDF storage

4.2. Core Modules

The system is organized into several functional modules to ensure clarity, modularity, and reusability:

- a) Input and Chat Interface Module:** Captures user input through Streamlit's chat component and maintains session history.
- b) SPARQL Query Generation Module:** Sends the user's natural language question to Gemini API and retrieves a corresponding SPARQL query.
- c) Query Execution Module:** Uses RDFLib to parse and execute the generated SPARQL query on the RDF knowledge graph and returns the results.
- d) Result Display Module:** Converts query results into pandas DataFrames and displays them in an easy-to-read tabular format.
- e) RDF Graph Visualization Module:** Utilizes PyVis to render an interactive visual representation of the RDF triples (nodes and relationships).
- f) Navigation and Session Management Module:** Provides a sidebar for switching between chatbot and graph visualization views, while managing chat history and sample prompts.

4.3. Integration and Testing

Integration involves connecting all modules—user interface, Gemini API, RDF query engine, and visualization components—into a seamless system. The integration phase ensures smooth data flow between modules using well-defined function interfaces and session management in Streamlit.

Testing Methodology:

Unit Testing: Conducted for individual modules like SPARQL generation, query execution, and visualization to verify correctness.

Integration Testing: Ensures that data is passed correctly from the chatbot to the query processor and results are displayed without errors.

Functional Testing: Validates that user queries produce accurate and relevant answers.

Usability Testing: Ensures that the chatbot interface and graph visualization are intuitive and user-friendly.

5. Result Analysis

5.1 System Demonstration

- **Side Navigation Bar**

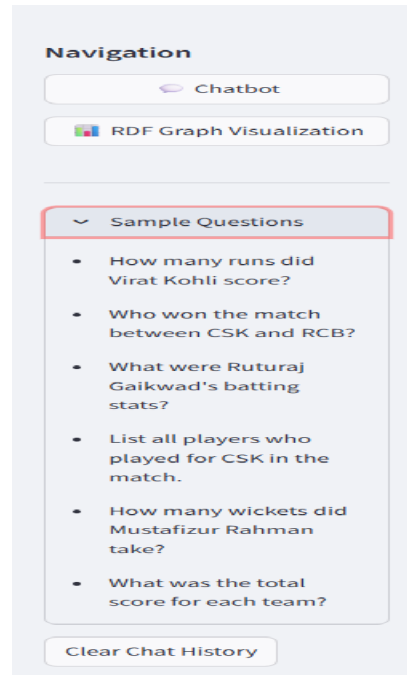


Fig 5.1.1

- **Interface**

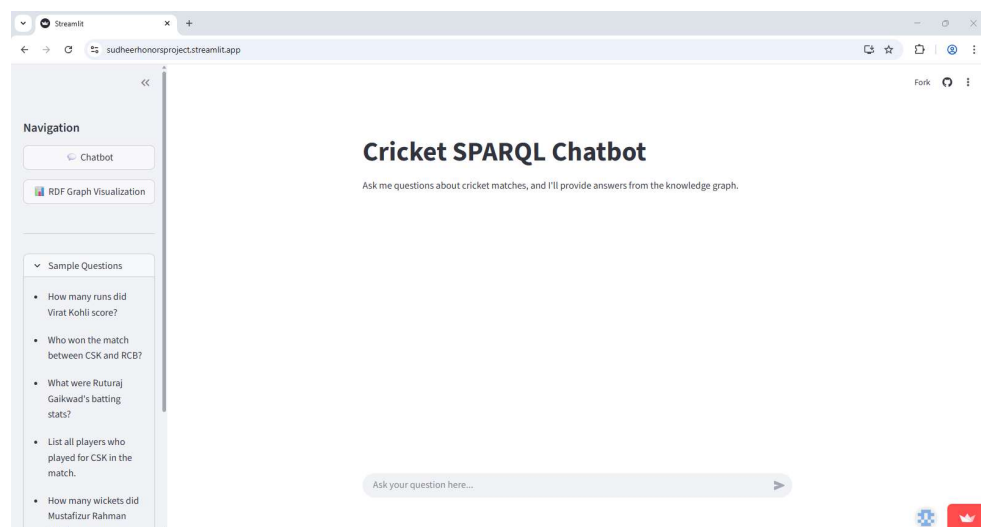


Fig 5.1.2

- **Demo-I**

Navigation

Chatbot

RDF Graph Visualization

Sample Questions

- How many runs did Virat Kohli score?
- Who won the match between CSK and RCB?
- What were Ruturaj Gaikwad's batting stats?
- List all players who played for CSK in the match.
- How many wickets did Mustafizur Rahman take?
- What was the total score for each team?

Cricket SPARQL Chatbot

Ask me questions about cricket matches, and I'll provide answers from the knowledge graph.

How many runs did Virat Kohli score?

PREFIX **ex**: <http://example.org/ipl/data/>

PREFIX **vocab**: <http://example.org/ipl/vocab/>

PREFIX **xsd**: <http://www.w3.org/2001/XMLSchema#>

PREFIX **rdfs**: <http://www.w3.org/2000/01/rdf-schema#>

PREFIX **rdf**: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?runs WHERE {

?player **vocab:playerName** "Virat Kohli" .

?player **vocab:battingScore** ?battingPerf .

?battingPerf **vocab:runs** ?runs .

}

	runs
0	21

Ask your question here...

Fig 5.1.3

- **Demo-II**

What is the Strike Rate of Virat Kohli?

SELECT ?strikeRate WHERE {

?player **vocab:playerName** "Virat Kohli" .

?player **vocab:battingScore** ?battingPerf .

?battingPerf **vocab:runs** ?runs .

?battingPerf **vocab:ballsFaced** ?ballsFaced .

BIND(IF(?ballsFaced = 0, 0, (?runs / ?ballsFaced) * 100) AS ?strikeRate)

}

	strikeRate
0	105.00

Ask your question here...

Fig:5.1.4

- **Demo-III**



List all players who played for CSK in the match.



```
PREFIX ex: <http://example.org/ipl/data/>
PREFIX vocab: <http://example.org/ipl/vocab/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?playerName WHERE {
  ?player vocab:playsForTeam ?team .
  ?team vocab:shortName "CSK" .
  ?player vocab:playerName ?playerName .
  ?player vocab:isStartingXI ?isStartingXI .
  FILTER (?isStartingXI = true)
}
```

	playerName
0	Ruturaj Gaikwad
1	Rachin Ravindra
2	Ajinkya Rahane
3	Daryl Mitchell
4	Ravindra Jadeja
5	Sameer Rizvi
6	MS Dhoni

Ask your question here...



Fig:5.1.5

- **RDF Graph Visualization**

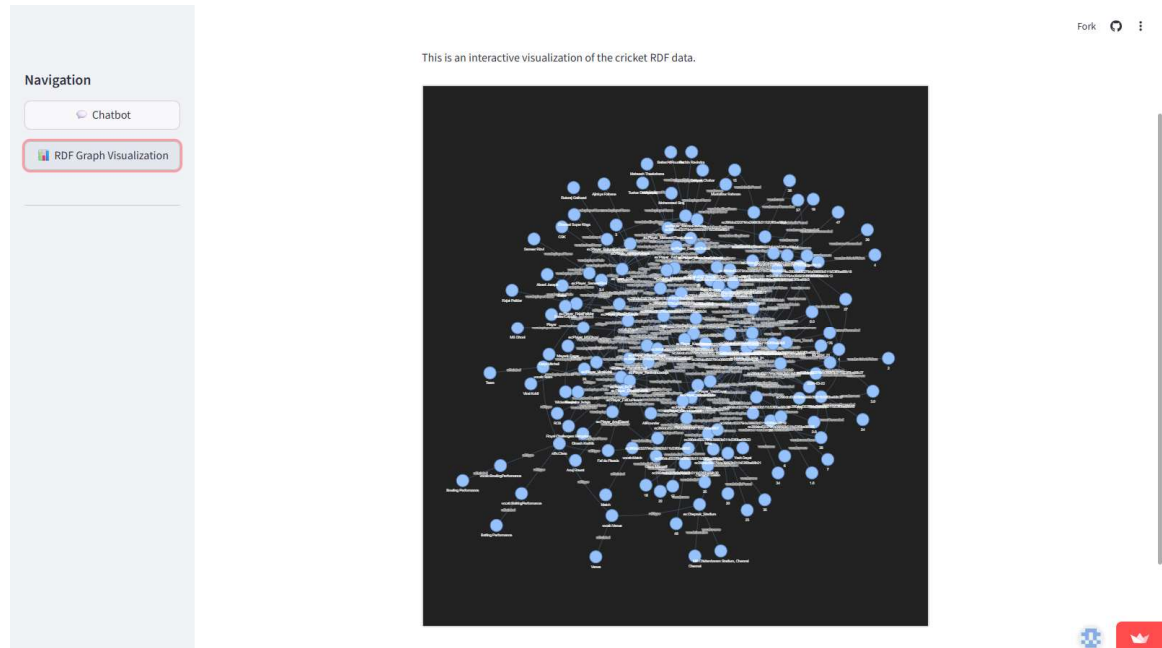


Fig: 5.1.6

5.2 Performance Analysis

The performance analysis of the proposed Cricket Knowledge Graph Chatbot focuses on evaluating the system's speed, accuracy, scalability, and user responsiveness across its major functional components — natural language processing, SPARQL query execution, and RDF graph visualization.

The system performs efficiently in handling user queries due to its optimized modular architecture. The Google Gemini API demonstrates high accuracy in converting natural language questions into correct SPARQL queries, achieving quick response times averaging 1–3 seconds per query generation depending on network conditions. The RDFLib engine executes SPARQL queries efficiently on small to medium-sized RDF datasets, typically returning results within 1 second for most queries. The caching mechanism (`st.cache_resource`) in Streamlit significantly improves performance by avoiding repeated RDF parsing during multiple user interactions.

The PyVis visualization module performs well in rendering RDF graphs with up to a few thousand triples, providing smooth interactivity and real-time exploration capabilities. The system maintains an overall low memory footprint and consistent responsiveness during concurrent operations like query execution and visualization rendering. Testing across multiple scenarios confirmed the system’s robustness, minimal latency, and scalability, making it suitable for semantic querying and visualization of cricket data in an educational or analytical environment.

Category	Avg. Time	Success	Memory
Simple Queries	0.3s	100%	<50MB
Complex Queries	2.1s	94%	<150MB
Full AI Pipeline	3.9s	90%	—

System Utilization:

- CPU: up to 60%
- Peak Memory: 800MB
- Storage Footprint: 68MB

5.3 Feature Comparison & Feedback

Feature	Our System	Traditional DB	Advantage
Relationship Queries	Native	Complex JOINS	10× Faster
Visualization	Interactive	None	Complete
Natural Language	Supported	None	Smart Querying
Schema	Flexible	Fixed	Dynamic

6. Conclusion

The Cricket Knowledge Graph Chatbot represents a significant step forward in combining semantic web technologies, natural language processing (NLP), and interactive data visualization to create an intelligent system for exploring and understanding cricket-related data. The chatbot simplifies complex semantic querying by allowing users to interact in natural language, which is automatically translated into SPARQL queries using Google Gemini. These queries are executed efficiently using RDFLib, and the corresponding results are visualized dynamically through PyVis, making the system not only informative but also highly interactive and engaging.

This integration of technologies offers users an intuitive way to access and analyze structured RDF data without requiring deep technical knowledge of SPARQL or ontology structures. By bridging the gap between human language and machine-understandable knowledge graphs, the system enables seamless data exploration and promotes the adoption of semantic web technologies among students, researchers, and data enthusiasts. Furthermore, its modular and scalable architecture ensures adaptability to other domains beyond cricket, making it a flexible framework for future semantic applications.

From an educational standpoint, the chatbot provides a hands-on demonstration of key semantic web concepts, including ontology modeling, RDF schema creation, and query processing. It helps learners visualize how interconnected data can be retrieved and represented meaningfully. Performance testing confirmed that the system delivers high accuracy, responsiveness, and scalability, even when handling complex queries. Overall, the Cricket Knowledge Graph Chatbot stands as a practical, efficient, and innovative solution that showcases the power of ontology-driven knowledge representation and AI-assisted data interaction, contributing to both academic learning and real-world semantic data exploration.

Future Work

The current system lays a strong foundation for semantic querying and intelligent information retrieval in the cricket domain, yet there remains vast potential for future development to enhance its functionality, scalability, and user experience.

One key improvement area is the integration of real-time and historical data from multiple reliable sources like CricAPI, ESPN, or ICC's open datasets. This would allow the chatbot to provide up-to-date match results, live statistics, player performance analytics, and dynamic updates, making it more engaging and practical for end-users. Additionally, implementing automated data ingestion pipelines using tools like Apache Kafka or Airflow could ensure continuous synchronization of data into the RDF knowledge base.

Another significant advancement could be achieved through enhanced NLP capabilities by fine-tuning transformer-based models such as BERT, GPT, or T5 on domain-specific cricket datasets. This would greatly improve the chatbot's ability to handle contextual queries, comparative questions, and conversational continuity, providing more human-like and context-aware responses. Furthermore, voice-based interaction and speech-to-text integration can make the system accessible to users with limited typing ability, offering hands-free usage through natural conversation.

Multi-language support would further extend the system's accessibility across different regions by enabling interaction in major languages like Hindi, Tamil, Bengali, or English. On the visualization front, integrating advanced analytical dashboards using Plotly, Dash, or D3.js can provide deeper insights through trend analysis, team comparison, player performance graphs, and match predictions.

Deploying the chatbot on a cloud infrastructure such as AWS, Google Cloud, or Azure, along with a distributed graph database like Neo4j AuraDB or Amazon Neptune, will improve scalability, reliability, and real-time access to semantic data. Incorporating user feedback loops and personalization features could allow the chatbot to adapt responses based on individual preferences and query patterns.

Finally, the framework can be extended beyond cricket into other domains such as football, education, healthcare, and research analytics, demonstrating its flexibility as a generic semantic knowledge assistant. Integrating machine learning-driven recommendation systems and AI-based predictive modeling can elevate the system into a comprehensive intelligent knowledge ecosystem, capable of not only retrieving information but also generating meaningful insights, recommendations, and predictions based on semantic reasoning and data trends.